

INSERTION AT THE BEGINNING, AT THE END, AND AT THE MIDDLE:

```
#include <iostream>

using namespace std;

class Node {
public:
    int data;
    Node* prev;
    Node* next;

    Node(int value) {
        data = value;
        prev = nullptr;
        next = nullptr;
    }
};

int main() {
    // Create the first node (head of the list)
    Node* head = new Node(10);

    // Create and link the second node
    head->next = new Node(20);
    head->next->prev = head;

    // Create and link the third node
    head->next->next = new Node(30);
    head->next->next->prev = head->next;

    // Create and link the fourth node
    head->next->next->next = new Node(50);
    head->next->next->next->prev = head->next->next;

    //Insertion at the beginning:
    Node* new_head = new Node(0);
    new_head->next = head;
    head = new_head;
    cout << "INSERTION" << endl;
    cout << "Insert: " << head->data << endl;
```

```

// Insertion at the end
// Traverse the list forward
Node* new_tail;
Node* temp_tail = head;
while (temp_tail != nullptr) {
    if (temp_tail->next == nullptr) {
        new_tail = new Node(60);
        temp_tail -> next = new_tail;
        new_tail -> prev = temp_tail;
        cout << "Insert: " << new_tail -> data << endl;
        break;
    }
    // Move to the next node
    temp_tail = temp_tail->next;
}

//Insertion at the middle
Node* new_mid = new Node(40);
Node* temp_mid = head;
int index = 4;
int i = 0; //loop variable
while(i < index-1){
    temp_mid = temp_mid -> next;
    i++;
}
new_mid -> next = temp_mid -> next ;
temp_mid -> next = new_mid;
new_mid -> prev = temp_mid;
cout << "Insert: " << new_mid -> data << endl;

// Traverse the list forward and print elements
Node* temp = head;
while (temp != nullptr) {
    cout << temp->data;

    // Print the arrow only if there is a next node
    if (temp->next != nullptr) {
        cout << " <-> ";
    }
    // Move to the next node
    temp = temp->next;
}

```

```
cout << endl;
```

```
//Deletion at the beginning:  
cout << "DELETION" << endl;  
Node* temp_head = head -> next;  
delete head;  
head = temp_head;  
cout << "Delete: 0" << endl;
```

```
//Deletion at the middle  
Node* del_mid;  
Node* temp_var = head;  
int ind = 3;  
int j = 0; //loop variable  
while(j < ind-1){  
    temp_var = temp_var -> next;  
    j++;  
}  
cout << "Delete: 40 " << endl;  
del_mid = temp_var -> next;  
temp_var -> prev = del_mid -> prev;  
temp_var -> next = del_mid -> next;  
delete del_mid;
```

```
// Deletion at the end  
Node* tail;  
Node* temp_variable = head;  
while (temp_variable != nullptr) {  
    if (temp_variable->next == nullptr) {  
        cout << "Delete: "<< temp_variable ->data << endl;  
        tail = temp_variable -> prev;  
        tail -> next = nullptr;  
        delete temp_variable;  
        break;  
    }  
    // Move to the next node  
    temp_variable = temp_variable->next;  
}
```

```
// Traverse the list forward and print elements  
Node* temp_t = head;
```

```

while (temp_t != nullptr) {
    cout << temp_t->data;

    // Print the arrow only if there is a next node
    if (temp_t->next != nullptr) {
        cout << " <-> ";
    }
    // Move to the next node
    temp_t = temp_t->next;
}
cout << endl;

// Clean up memory (Good practice!)
temp = head;
while (temp != nullptr) {
    Node* nextNode = temp->next;
    delete temp;
    temp = nextNode;
}

return 0;
}

```

The screenshot shows a C++ IDE with a file named `main.cpp`. The code implements a linked list with insertion and deletion operations. The output window shows the following sequence of events:

```

102     j++;
103 }
104 cout << "Delete: 40 " << endl;
105 del_mid = temp_var -> next;
106 temp_var -> prev = del_mid -> prev;
107 temp_var -> next = del_mid -> next;
108 delete del_mid;
109
110 // Deletion at the end
111 Node* tail;
112 Node* temp_variable = head;
113 while (temp_variable != nullptr) {
114     if (temp_variable->next == nullptr) {
115         cout << "Delete: " << temp_variable->data << endl;
116         tail = temp_variable -> prev;
117         tail -> next = nullptr;
118         delete temp_variable;
119         break;
120     }
121     // Move to the next node
122     temp_variable = temp_variable->next;

```

The output window displays the following text:

```

^
INSERTION
Insert: 0
Insert: 60
Insert: 40
0 <-> 10 <-> 20 <-> 30 <-> 40 <-> 50 <-> 60
DELETION
Delete: 0
Delete: 40
Delete: 60
10 <-> 20 <-> 30 <-> 50

=== Code Execution Successful ===

```